

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px

from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import MinMaxScaler
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.cluster import KMeans
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import precision_score, recall_score, f1_score
from sklearn.metrics import roc_curve, auc, ConfusionMatrixDisplay
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.pipeline import Pipeline
from sklearn.metrics import precision_score, recall_score, f1_score,
average_precision_score
from sklearn.model_selection import train_test_split

from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive

file_path1 = '/content/drive/My Drive/MusicData/data.csv'
file_path2 = '/content/drive/My Drive/MusicData/data_by_genres.csv'
file_path3 = '/content/drive/My Drive/MusicData/data_by_year.csv'

df = pd.read_csv(file_path1)
df_genres = pd.read_csv(file_path2)
df_year = pd.read_csv(file_path3)

df.head()

{"type": "dataframe", "variable_name": "df"}

```

## Data Collection and Preprocessing

Handle Missing or Inconsistent Data or Cheaking All Null Values

Here I use Median because Robust to Outliers, Skewed Data(Long tail), Irregular Data and handle categorical columns using mode

```

numerical_cols = df.select_dtypes(include=['float64',
'int64']).columns

```

```

for col in numerical_cols:
    skewness = df[col].skew()
    print(f"Skewness for {col}: {skewness}")
    if -0.5 <= skewness <= 0.5:
        df[col].fillna(df[col].mean(), inplace=True)
        print(f"Filling missing values in {col} with the mean.")
    else:
        df[col].fillna(df[col].median(), inplace=True)
        print(f"Filling missing values in {col} with the median.")

categorical_cols = df.select_dtypes(include=['object']).columns
for col in categorical_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)
df.info()

```

## Encode categorical variables using label encoding

```

categorical_cols = df.select_dtypes(include=['object']).columns
label_encoder = LabelEncoder()

```

```

for col in categorical_cols:
    df[col] = label_encoder.fit_transform(df[col])

```

```
print(df.head())
```

|          | valence | year | acousticness | artists | danceability | duration_ms |
|----------|---------|------|--------------|---------|--------------|-------------|
| energy \ |         |      |              |         |              |             |
| 0        | 0.0594  | 1921 | 0.982        | 26839   | 0.279        | 831667      |
| 0.211    |         |      |              |         |              |             |
| 1        | 0.9630  | 1921 | 0.732        | 7382    | 0.819        | 180533      |
| 0.341    |         |      |              |         |              |             |
| 2        | 0.0394  | 1921 | 0.961        | 16378   | 0.328        | 500062      |
| 0.166    |         |      |              |         |              |             |
| 3        | 0.1650  | 1921 | 0.967        | 10077   | 0.275        | 210000      |
| 0.309    |         |      |              |         |              |             |
| 4        | 0.2530  | 1921 | 0.957        | 23719   | 0.418        | 166693      |
| 0.193    |         |      |              |         |              |             |

|        | explicit | id     | instrumentalness | key | liveness | loudness | mode |
|--------|----------|--------|------------------|-----|----------|----------|------|
| name \ |          |        |                  |     |          |          |      |
| 0      | 0        | 96623  | 0.878000         | 10  | 0.665    | -20.096  | 1    |
| 83631  |          |        |                  |     |          |          |      |
| 1      | 0        | 169794 | 0.000000         | 7   | 0.160    | -12.441  | 1    |
| 20291  |          |        |                  |     |          |          |      |
| 2      | 0        | 43559  | 0.913000         | 3   | 0.101    | -14.850  | 1    |
| 38094  |          |        |                  |     |          |          |      |
| 3      | 0        | 85809  | 0.000028         | 5   | 0.381    | -9.316   | 1    |
| 24147  |          |        |                  |     |          |          |      |
| 4      | 0        | 105991 | 0.000002         | 3   | 0.229    | -10.096  | 1    |
| 123247 |          |        |                  |     |          |          |      |

|   | popularity | release_date | speechiness | tempo   |
|---|------------|--------------|-------------|---------|
| 0 | 4          | 0            | 0.0366      | 80.954  |
| 1 | 5          | 0            | 0.4150      | 60.936  |
| 2 | 5          | 0            | 0.0339      | 110.339 |
| 3 | 3          | 0            | 0.0354      | 100.109 |
| 4 | 2          | 0            | 0.0380      | 101.665 |

## Normalize All Numeric Features

I use MinMaxScaler to scale numerical features to a fixed range (generally 0 to 1)

```
numeric_cols = df.select_dtypes(include=np.number).columns
scaler = MinMaxScaler()
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])
print(f"Normalized {numeric_cols}")

Normalized Index(['valence', 'year', 'acousticness', 'artists',
                  'danceability',
                  'duration_ms', 'energy', 'explicit', 'id', 'instrumentalness',
                  'key',
                  'liveness', 'loudness', 'mode', 'name', 'popularity',
                  'release_date',
                  'speechiness', 'tempo'],
                  dtype='object')
```

Removes features with high percentage of a single value or zero variance

```
def remove_garbage_features(df):
    garbage_features = []

    for col in df.columns:
        if pd.api.types.is_numeric_dtype(df[col]):
            if df[col].value_counts(normalize=True).max() > 0.95:
                garbage_features.append(col)
            if df[col].std() == 0:
                garbage_features.append(col)

    print("Garbage features to remove:",garbage_features)
    df_cleaned = df.drop(columns=garbage_features)

    return df_cleaned

df = remove_garbage_features(df)
print(df.head())
```

## Perform Principal Component Analysis (PCA)

```
numeric_cols = df.select_dtypes(include=np.number).columns
scaler = StandardScaler()
df_scaled = pd.DataFrame(scaler.fit_transform(df[numeric_cols]),
                          columns=numeric_cols)

pca = PCA()
pca.fit(df_scaled)

explained_variance_ratio = pca.explained_variance_ratio_
cumulative_variance_ratio = np.cumsum(explained_variance_ratio)

n_components = np.argmax(cumulative_variance_ratio >= 0.95) + 1

print(f"Number of components explaining 95% variance: {n_components}")

pca = PCA(n_components=n_components)
df_pca = pca.fit_transform(df_scaled)

df_pca = pd.DataFrame(data=df_pca, columns=[f'PC{i+1}' for i in
                                           range(n_components)])

print(df_pca.head())

plt.figure(figsize=(8, 6))
plt.plot(range(1, len(explained_variance_ratio) + 1),
         cumulative_variance_ratio, marker='o')
plt.xlabel('Number of Principal Components')
plt.ylabel('Cumulative Explained Variance')
plt.title('Explained Variance Ratio vs. Number of Components')
plt.grid(True)
plt.show()
```

Number of components explaining 95% variance: 15

|        | PC1       | PC2       | PC3       | PC4       | PC5       | PC6       |
|--------|-----------|-----------|-----------|-----------|-----------|-----------|
| PC7 \  |           |           |           |           |           |           |
| 0      | -4.297586 | -2.305962 | 1.290642  | 3.769907  | -1.273376 | -0.223552 |
|        | 0.867248  |           |           |           |           |           |
| 1      | -1.892878 | 3.138542  | -0.568652 | -1.460478 | -0.504517 | -1.674225 |
|        | 1.998079  |           |           |           |           |           |
| 2      | -3.944837 | -1.961346 | 0.055729  | 0.794937  | 0.095130  | -0.481013 |
|        | 0.664904  |           |           |           |           |           |
| 3      | -2.828501 | -0.411011 | -0.694752 | 1.133900  | 0.296902  | -1.553448 |
|        | 0.309541  |           |           |           |           |           |
| 4      | -2.879947 | 0.423625  | -0.076866 | 0.633519  | 1.346278  | 1.003469  |
|        | 0.499128  |           |           |           |           |           |
|        | PC8       | PC9       | PC10      | PC11      | PC12      | PC13      |
| PC14 \ |           |           |           |           |           |           |
| 0      | 1.857401  | 1.444023  | 0.712500  | 2.329972  | 1.277172  | 0.966999  |

```

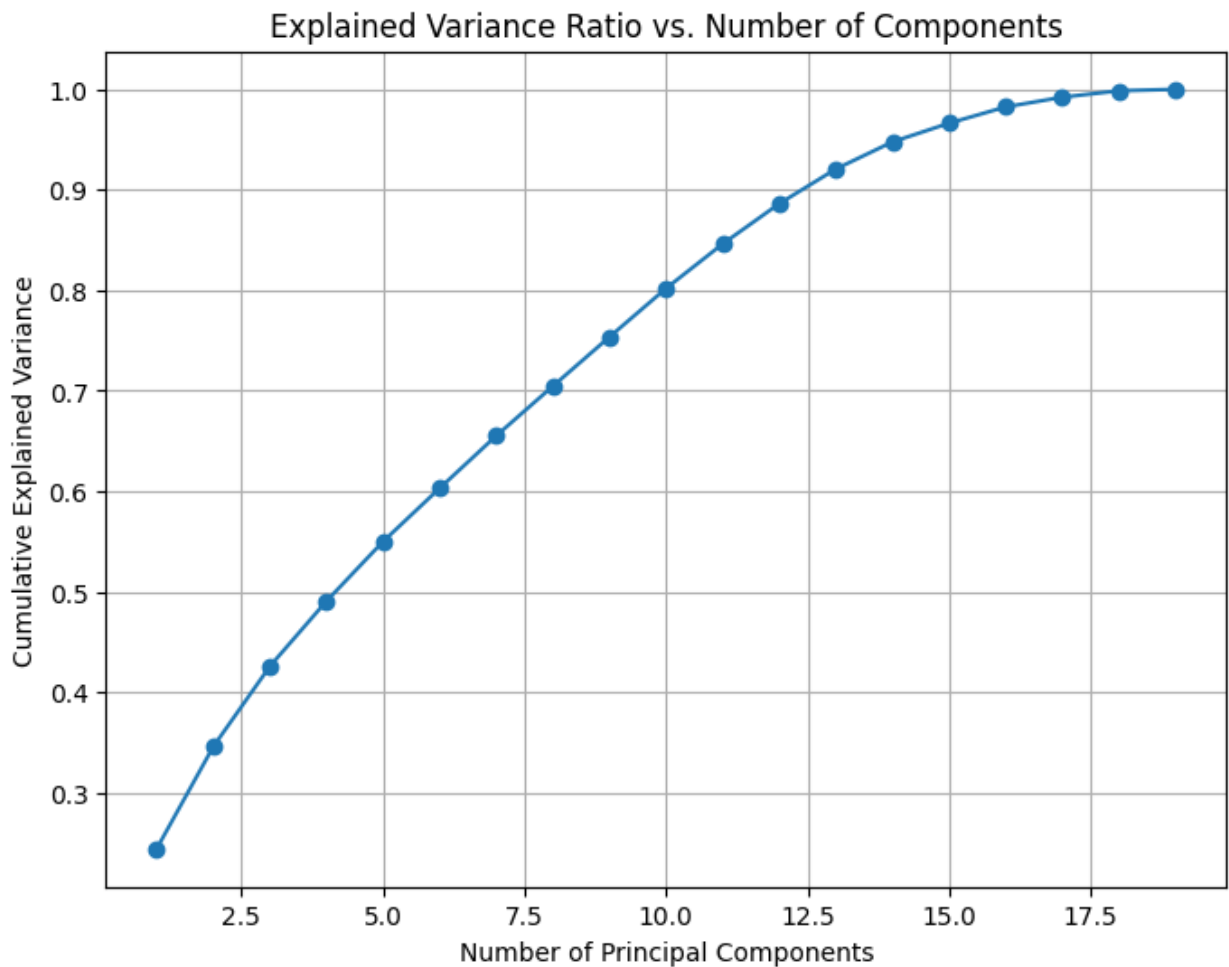
0.843704
1  0.758852  0.543259  0.904670  0.969998 -0.093689 -0.593885 -
0.784900
2  1.212145 -0.108431 -1.165764  1.364136  0.669314 -0.910227
0.469394
3 -0.430384 -0.399206  1.022902 -0.150026 -1.027497 -1.206530
0.778283
4 -0.309416  0.760815  0.613466 -0.510854 -1.021139 -0.772335
0.761587

```

```

PC15
0  0.044785
1  0.396960
2  0.692476
3  0.509608
4  0.579943

```



```

pca = PCA(n_components=2)
df_pca = pca.fit_transform(df_scaled)

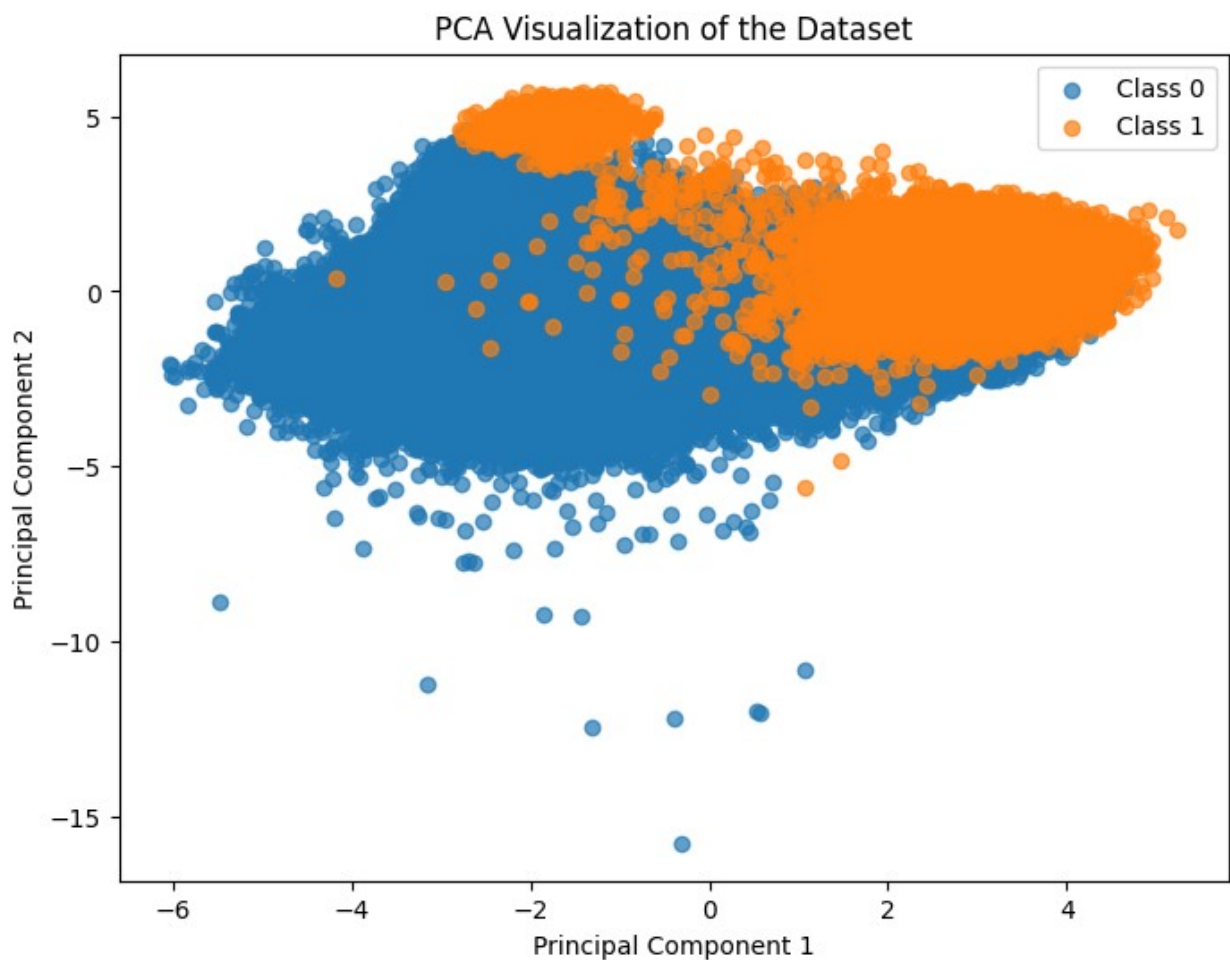
```

```

df_pca = pd.DataFrame(data=df_pca, columns=['PC1', 'PC2'])
df_pca['target'] = df['explicit']
y = df['explicit']

plt.figure(figsize=(8, 6))
plt.scatter(df_pca[y == 0]['PC1'], df_pca[y == 0]['PC2'], label='Class 0', alpha=0.7)
plt.scatter(df_pca[y == 1]['PC1'], df_pca[y == 1]['PC2'], label='Class 1', alpha=0.7)
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('PCA Visualization of the Dataset')
plt.legend()
plt.show()

```



# Dataset Characteristics and Exploratory Data Analysis

## Predict Model Report for Test Dataset

```
X = df_pca.drop('target', axis=1)
y = df['explicit']

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# Logistic Regression
logreg = LogisticRegression(max_iter=300)
logreg.fit(X_train, y_train)
y_pred_logreg = logreg.predict(X_test)
y_pred_prob_logreg = logreg.predict_proba(X_test)[: , 1]
print("Logistic Regression Classification Report:")
print(classification_report(y_test, y_pred_logreg))

# Support Vector Machine
svm = SVC(probability=True)
svm.fit(X_train, y_train)
y_pred_svm = svm.predict(X_test)
y_pred_prob_svm = svm.predict_proba(X_test)[: , 1]
print("SVM Classification Report:")
print(classification_report(y_test, y_pred_svm))
```

### Logistic Regression Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0.0          | 0.96      | 0.99   | 0.98     | 46805   |
| 1.0          | 0.82      | 0.60   | 0.69     | 4391    |
| accuracy     |           |        | 0.95     | 51196   |
| macro avg    | 0.89      | 0.79   | 0.83     | 51196   |
| weighted avg | 0.95      | 0.95   | 0.95     | 51196   |

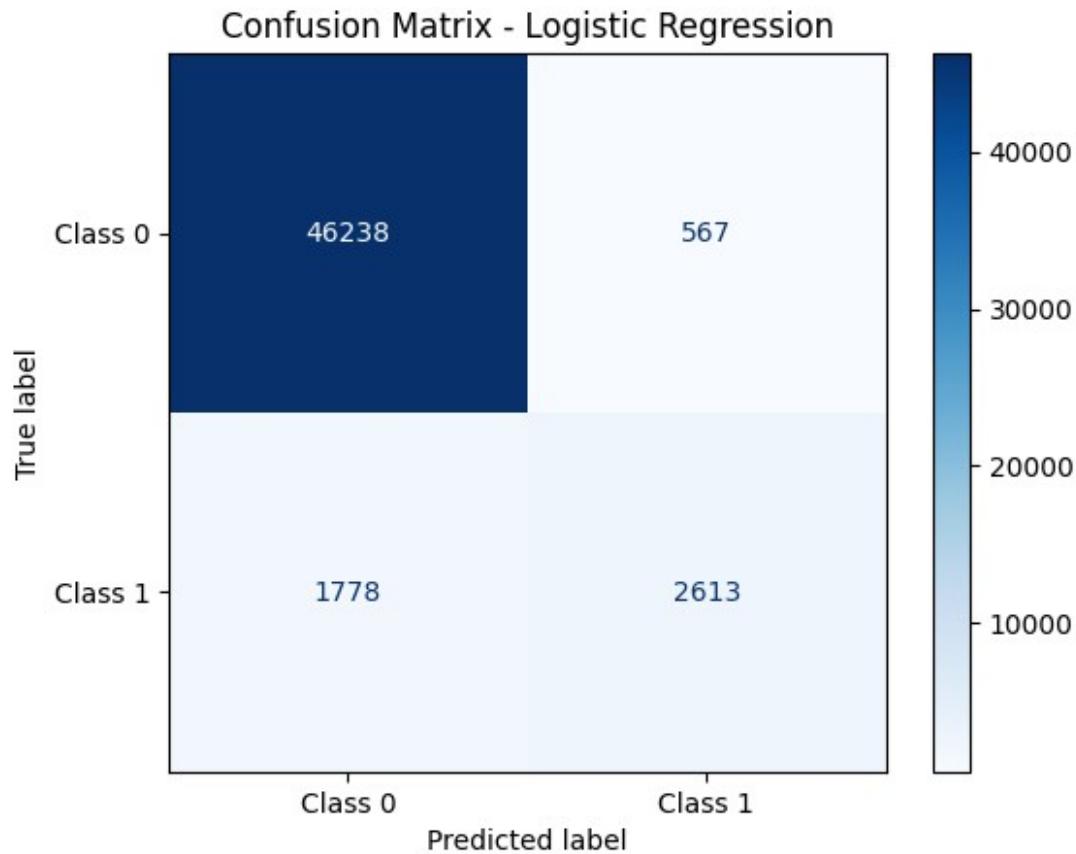
### SVM Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0.0          | 0.96      | 0.99   | 0.98     | 46805   |
| 1.0          | 0.89      | 0.56   | 0.69     | 4391    |
| accuracy     |           |        | 0.96     | 51196   |
| macro avg    | 0.92      | 0.78   | 0.83     | 51196   |
| weighted avg | 0.95      | 0.96   | 0.95     | 51196   |

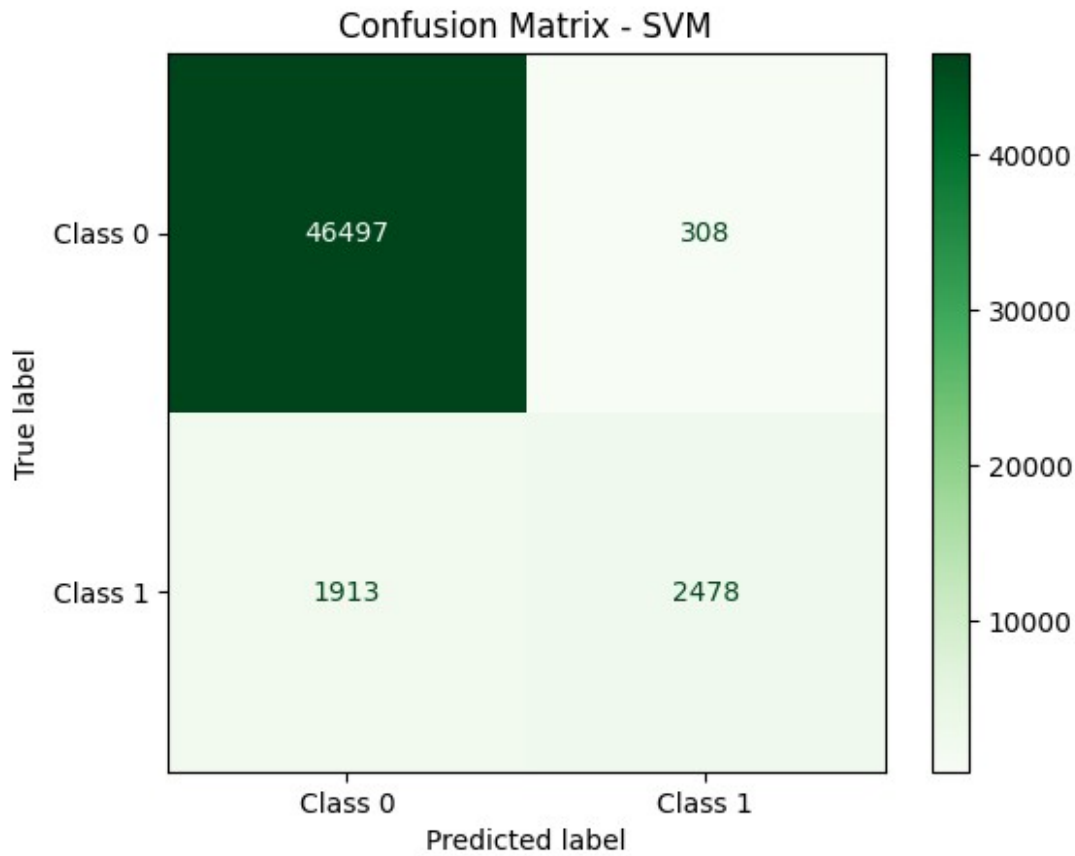
## Confusion Matrix

```
ConfusionMatrixDisplay.from_estimator(logreg, X_test, y_test,  
display_labels=["Class 0", "Class 1"], cmap="Blues")  
plt.title("Confusion Matrix - Logistic Regression")  
plt.show()
```

```
ConfusionMatrixDisplay.from_estimator(svm, X_test, y_test,  
display_labels=["Class 0", "Class 1"], cmap="Greens")  
plt.title("Confusion Matrix - SVM")  
plt.show()
```



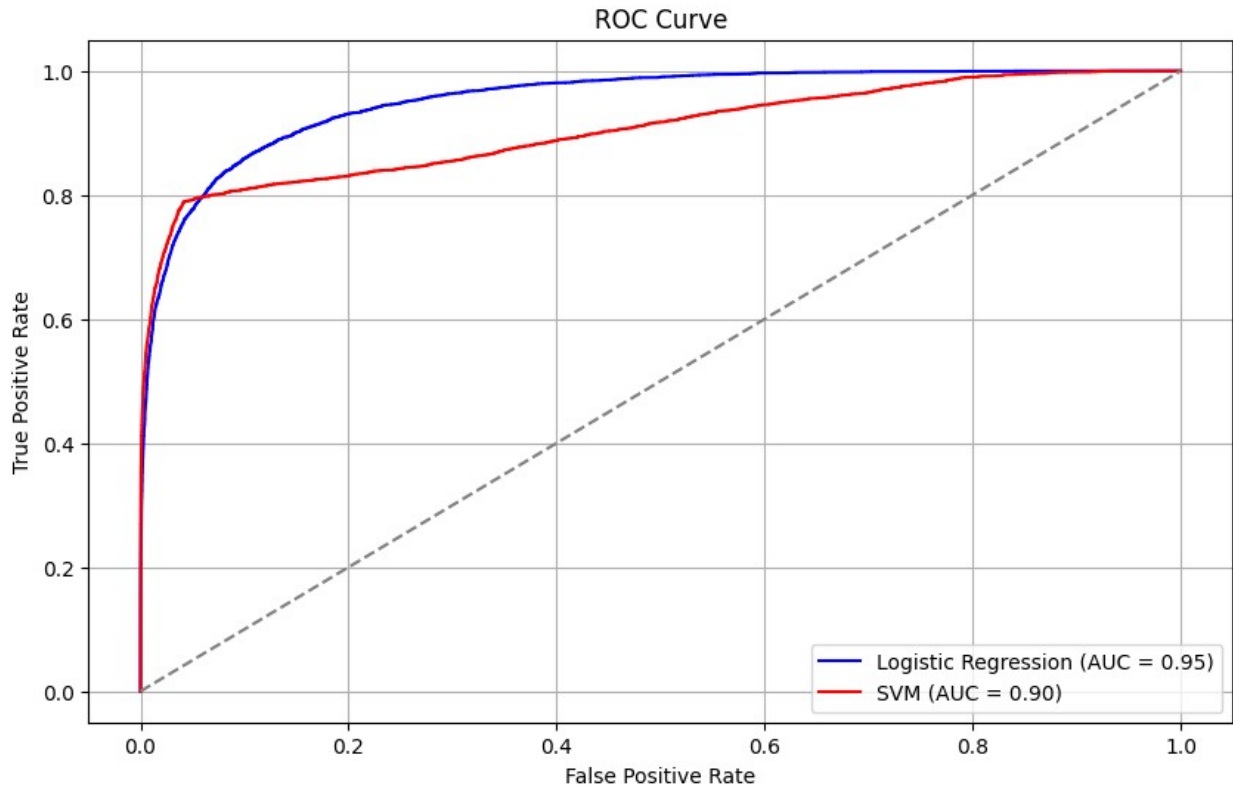




### ROC Curve (Receiver Operating characteristics)

```
fpr_logreg, tpr_logreg, _ = roc_curve(y_test, y_pred_prob_logreg)
roc_auc_logreg = auc(fpr_logreg, tpr_logreg)
fpr_svm, tpr_svm, _ = roc_curve(y_test, y_pred_prob_svm)
roc_auc_svm = auc(fpr_svm, tpr_svm)

plt.figure(figsize=(10, 6))
plt.plot(fpr_logreg, tpr_logreg, color='blue', label=f'Logistic
Regression (AUC = {roc_auc_logreg:.2f})')
plt.plot(fpr_svm, tpr_svm, color='red', label=f'SVM (AUC =
{roc_auc_svm:.2f})')
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.grid()
plt.show()
```



### Predict Model Report for Training Dataset

```
# Logistic Regression
y_pred_logreg_train = logreg.predict(X_train)
y_pred_prob_logreg_train = logreg.predict_proba(X_train)[: , 1]
print("Logistic Regression Classification Report :")
print(classification_report(y_train, y_pred_logreg_train))

# Support Vector Machine
y_pred_svm_train = svm.predict(X_train)
y_pred_prob_svm_train = svm.predict_proba(X_train)[: , 1]
print("SVM Classification Report :")
print(classification_report(y_train, y_pred_svm_train))
```

#### Logistic Regression Classification Report :

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0.0          | 0.96      | 0.99   | 0.97     | 109415  |
| 1.0          | 0.80      | 0.57   | 0.67     | 10042   |
| accuracy     |           |        | 0.95     | 119457  |
| macro avg    | 0.88      | 0.78   | 0.82     | 119457  |
| weighted avg | 0.95      | 0.95   | 0.95     | 119457  |

#### SVM Classification Report :

|  | precision | recall | f1-score | support |
|--|-----------|--------|----------|---------|
|--|-----------|--------|----------|---------|

|              |     |      |      |      |        |
|--------------|-----|------|------|------|--------|
|              | 0.0 | 0.96 | 0.99 | 0.98 | 109415 |
|              | 1.0 | 0.88 | 0.54 | 0.67 | 10042  |
| accuracy     |     |      |      | 0.96 | 119457 |
| macro avg    |     | 0.92 | 0.77 | 0.82 | 119457 |
| weighted avg |     | 0.95 | 0.96 | 0.95 | 119457 |

### Comparison of Logistic Regression and SVM Performance on Training and Test Data Using Graphical Representation

```
def plot_classification_report(y_true, y_pred, model_name, data_type,
ax):
    metrics = {
        'Accuracy': accuracy_score(y_true, y_pred),
        'Precision': precision_score(y_true, y_pred),
        'Recall': recall_score(y_true, y_pred),
        'F1-score': f1_score(y_true, y_pred)
    }

    metrics_df = pd.DataFrame(list(metrics.items()),
columns=['Metric', 'Score'])

    sns.barplot(x='Metric', y='Score', hue='Metric', data=metrics_df,
palette="viridis", ax=ax, legend=False)

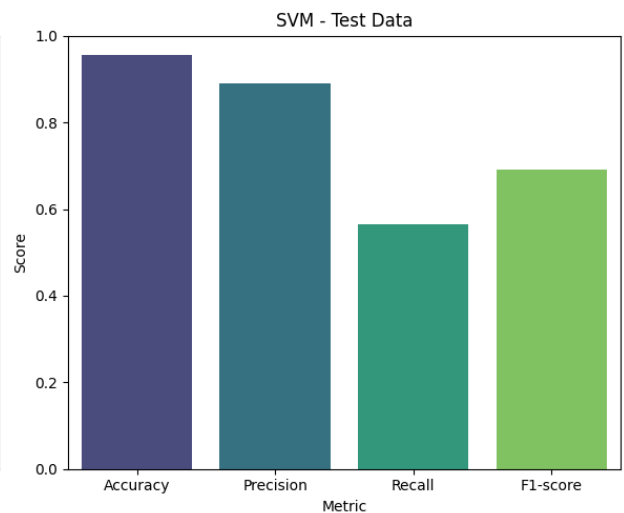
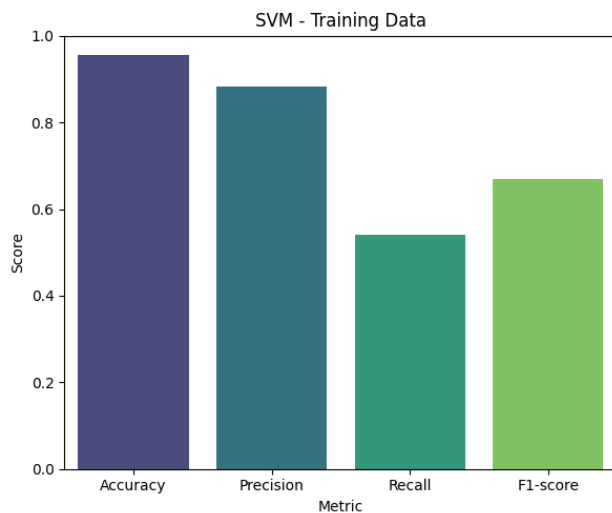
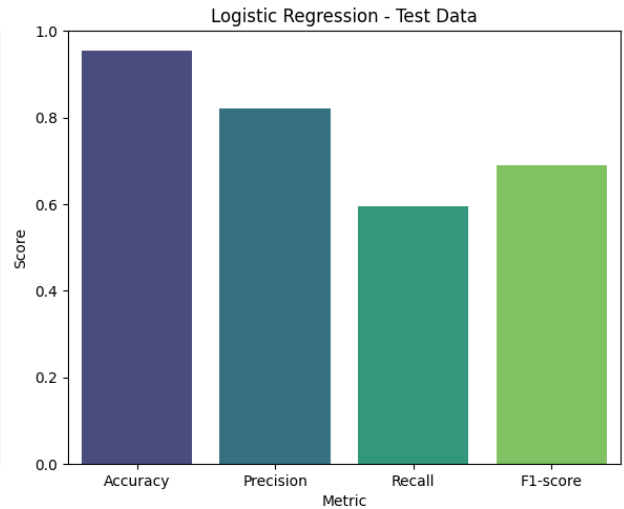
    ax.set_title(f"{model_name} - {data_type} Data")
    ax.set_ylim(0, 1)
    ax.set_ylabel("Score")

fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes = axes.flatten()

predictions = [
    (y_train, y_pred_logreg_train, 'Logistic Regression', 'Training'),
    (y_test, y_pred_logreg, 'Logistic Regression', 'Test'),
    (y_train, y_pred_svm_train, 'SVM', 'Training'),
    (y_test, y_pred_svm, 'SVM', 'Test')
]

for i, (y_true, y_pred, model_name, data_type) in
enumerate(predictions):
    plot_classification_report(y_true, y_pred, model_name, data_type,
axes[i])

plt.tight_layout()
plt.show()
```



**Descriptive statistics of all datasets** Count, mean, std, min, percentile, max

```
df.describe()
```

## Visualize Distributions of Key Features

```
!pip install dash
!pip install pyngrok
!pip install dash==2.11.0
!pip install jupyter_dash
!pip install dash-bootstrap-components

from dash import Dash, dcc, html, Input, Output, callback
import plotly.express as px
from jupyter_dash import JupyterDash
from dash import dcc, html
from dash.dependencies import Input, Output
from pyngrok import ngrok
```

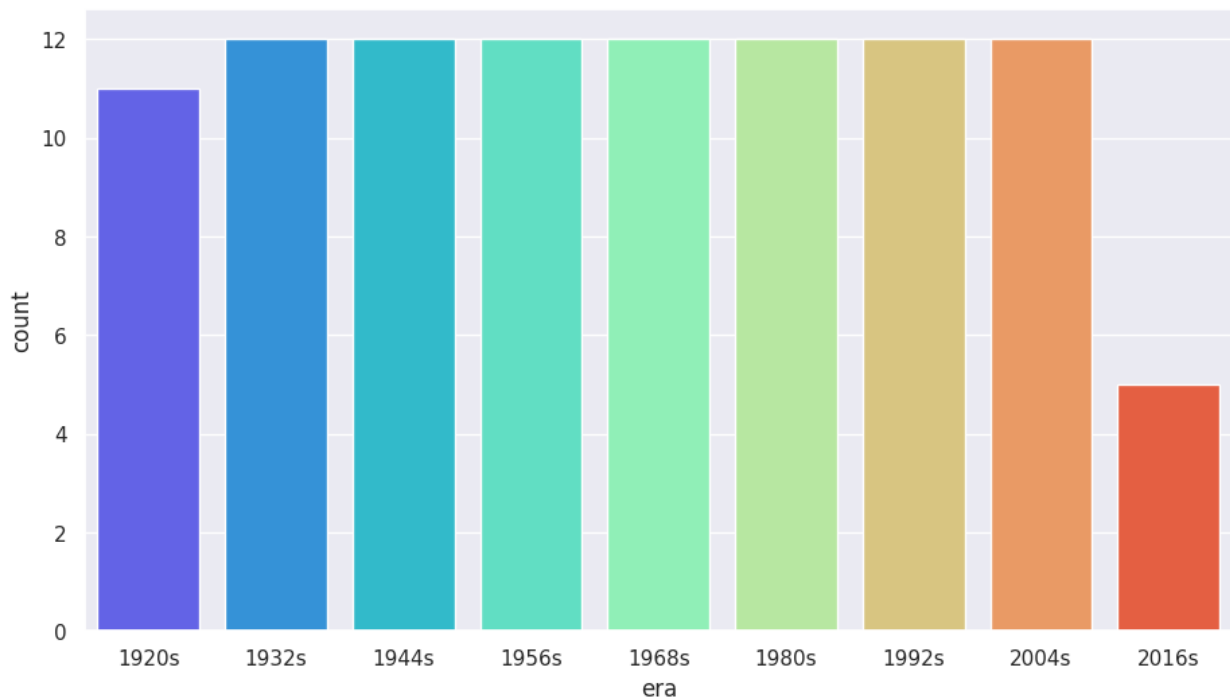
```
import plotly.graph_objects as go
import dash_bootstrap_components as dbc
```

**Genre Trends(data\_by\_genres): Top 10 genres by popularity : Grouped Bar plots for the energy levels, valence and so on**

```
top10_genres = df_genres.nlargest(10, 'popularity')
fig = px.bar(top10_genres, x='genres', y=['danceability', 'energy',
'liveness', 'speechiness', 'valence'], barmode='group')
fig.show()
```

**The plot shows the frequency(sound of music) of records in each 12-year interval**

```
df_year['era'] = (df_year['year'] // 12 * 12).astype(str) + 's'
sns.set(rc={'figure.figsize': (11, 6)})
sns.countplot(data=df_year, x='era', hue='era', palette='rainbow',
dodge=False, legend=False)
plt.show()
```



**Yearly Trends(data\_by\_year): Line plot trends of variables like popularity or danceability over time and so on**

```
features = ['acousticness', 'popularity', 'danceability', 'energy',
'instrumentalness', 'liveness', 'speechiness', 'valence', 'tempo']
feature_titles = ['Acousticness Trend', 'Popularity Trend',
```

```

'Danceability Trend', 'Energy Trend', 'Instrumentalness Trend',
'Liveness Trend', 'Speechiness Trend', 'Valence Trend', 'Tempo Trend']

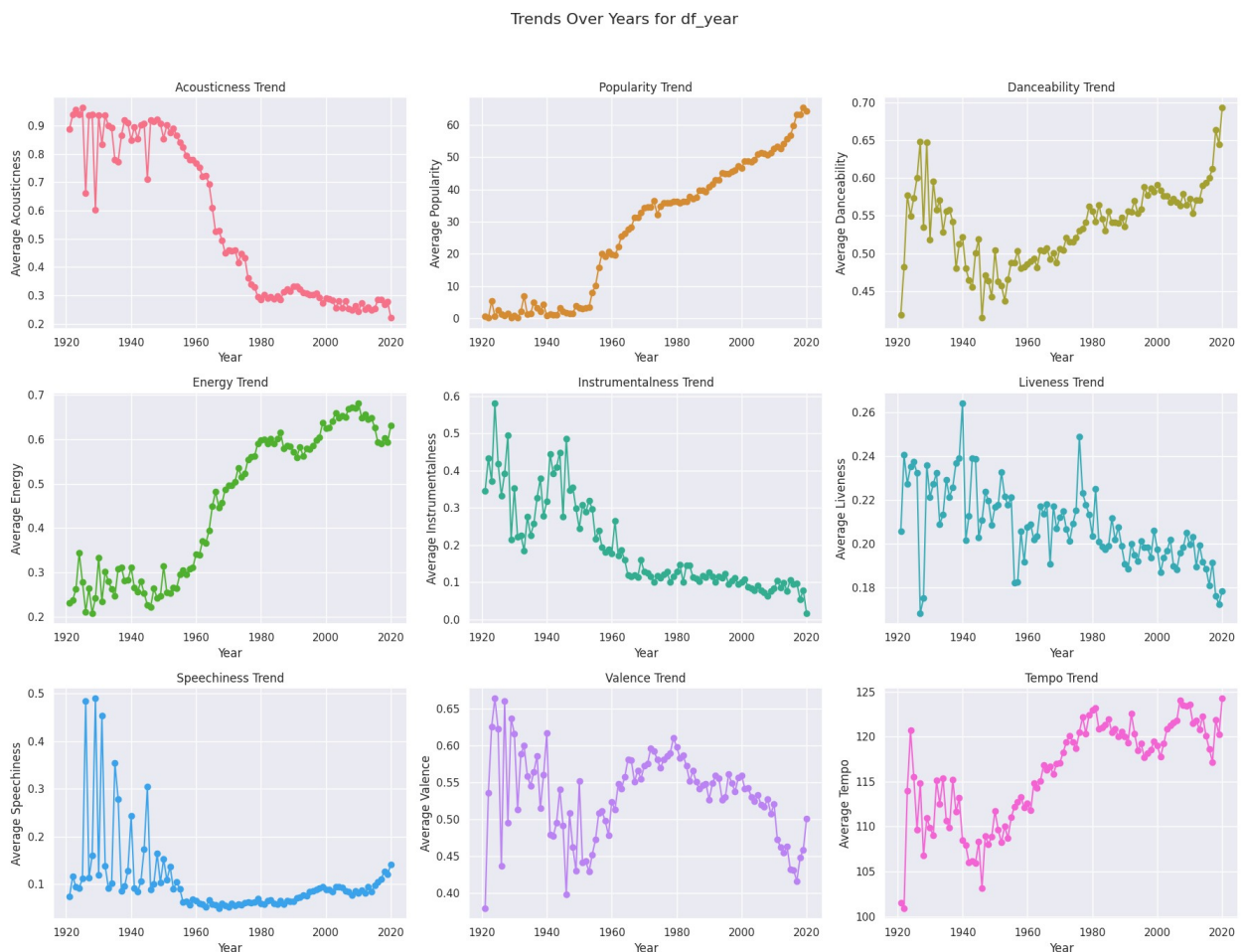
plt.figure(figsize=(18, 14))
plt.suptitle(f'Trends Over Years for df_year', fontsize=16)

palette = sns.color_palette("husl", len(features))

for i, (feature, title) in enumerate(zip(features, feature_titles)):
    if feature in df_year.columns:
        plt.subplot(3, 3, i + 1)
        yearly_trend = df_year.groupby('year')[feature].mean()
        plt.plot(yearly_trend.index, yearly_trend.values, marker='o',
color=palette[i])
        plt.xlabel('Year')
        plt.ylabel(f'Average {feature.capitalize()}')
        plt.title(title)
        plt.grid(True)

plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()

```



## Yearly Trends Dashboard : Line plot trends

```
!kill ngrok
ngrok.kill()

ngrok.set_auth_token("2rM2jNChPC6upKIyu1R0hywZIQe_5yJe2EtfCXUemirxV9aME")
app = JupyterDash(__name__)

features = ['acousticness', 'popularity', 'danceability', 'energy',
            'instrumentalness', 'liveness', 'speechiness', 'valence', 'tempo']
feature_titles = ['Acousticness Trend', 'Popularity Trend',
                  'Danceability Trend', 'Energy Trend', 'Instrumentalness Trend',
                  'Liveness Trend', 'Speechiness Trend', 'Valence Trend', 'Tempo Trend']

app.layout = html.Div([
    html.H1("Yearly Trends"),
    dcc.Dropdown(
        id='feature-dropdown',
        options=[{'label': title, 'value': feature} for feature, title
in zip(features, feature_titles)],
        value='popularity'
    ),
    dcc.Graph(id='yearly-trend-graph')
])

@app.callback(
    Output('yearly-trend-graph', 'figure'),
    Input('feature-dropdown', 'value')
)
def update_graph(selected_feature):
    if selected_feature in df_year.columns:
        yearly_trend = df_year.groupby('year')
[selected_feature].mean()
        fig = go.Figure(data=go.Scatter(x=yearly_trend.index,
y=yearly_trend.values, mode='lines+markers'))

        fig.update_layout(title=f'{feature_titles[features.index(selected_feature)]}',
                           xaxis_title='Year',
                           yaxis_title=f'Average
{selected_feature.capitalize()}')
        return fig
    else:
        return go.Figure()

public_url = ngrok.connect(8051)
print("Dash app is running at:", public_url)

app.run_server(debug=False)
```

```
WARNING:pyngrok.process.ngrok:t=2025-02-15T06:34:07+0000 lvl=warn  
msg="Stopping forwarder" name=http-8052-661a5b8f-079e-45db-aeba-  
2f6773000255 acceptErr="failed to accept connection: Listener closed"  
WARNING:pyngrok.process.ngrok:t=2025-02-15T06:34:07+0000 lvl=warn  
msg="Error restarting forwarder" name=http-8052-661a5b8f-079e-45db-  
aeba-2f6773000255 err="failed to start tunnel: session closed"
```

```
Dash app is running at: NgrokTunnel: "https://be9b-130-211-249-  
200.ngrok-free.app" -> "http://localhost:8051"
```

```
<IPython.core.display.Javascript object>
```

Dash app running on:

Warning: This function may stop working due to changes in browser security.

Try `serve\_kernel\_port\_as\_iframe` instead.

```
<IPython.core.display.Javascript object>
```

## Spotify Music Features Histogram Dashboard

```
from dash import Dash, dcc, html  
from dash.dependencies import Input, Output  
from pyngrok import ngrok  
from pyngrok import conf, installer  
import os  
import io  
from google.colab import files  
  
import requests  
from pyngrok.conf import PyngrokConfig  
  
ngrok.kill()  
  
ngrok.set_auth_token("2rM2jNChPC6upKIyu1R0hywZIQe_5yJe2EtfCXUemirxV9aME")  
  
datasets = {  
    "Dataset 1 (data.csv)": pd.read_csv('/content/drive/My  
Drive/MusicData/data.csv'),  
    "Dataset 3 (data_by_genres.csv)": pd.read_csv('/content/drive/My  
Drive/MusicData/data_by_genres.csv'),  
    "Dataset 4 (data_by_year.csv)": pd.read_csv('/content/drive/My  
Drive/MusicData/data_by_year.csv'),  
}  
  
features = ['acousticness', 'danceability', 'energy', 'liveness',  
'loudness', 'speechiness', 'tempo', 'valence', 'key']  
  
app = Dash(__name__)
```



```

app.layout = html.Div([
    html.H1("Spotify Music Features Histogram Dashboard",
style={'text-align': 'center'}),

    dcc.Dropdown(
        id='dropdown-dataset',
        options=[{'label': name, 'value': name} for name in
datasets.keys()],
        value="Dataset 1 (data.csv)",
        style={'width': '50%', 'margin': '0 auto'}
    ),

    dcc.Dropdown(
        id='dropdown-feature',
        options=[{'label': feature, 'value': feature} for feature in
features],
        value='acousticness',
        style={'width': '50%', 'margin': '20px auto'}
    ),

    dcc.Graph(id='histogram-plot'),
])

# Callback to update histogram based on dropdown selections
@app.callback(
    Output('histogram-plot', 'figure'),
    [Input('dropdown-dataset', 'value'),
    Input('dropdown-feature', 'value')]
)
def update_graph(selected_dataset, selected_feature):
    dataset = datasets[selected_dataset]

    if selected_feature in dataset.columns:
        fig = px.histogram(
            dataset,
            x=selected_feature,
            nbins=30, # Number of bins
            title=f"Distribution of {selected_feature} in
{selected_dataset}",
            template='plotly_dark'
        )
        fig.update_layout(xaxis_title=selected_feature,
yaxis_title='Count')
    else:
        # If feature is not found, return an empty figure
        fig = px.histogram(title=f"{selected_feature} not found in
{selected_dataset}")

    return fig

```

```
public_url = ngrok.connect(8050)
print("Dash app is running at:", public_url)

app.run_server(debug=False)

Dash app is running at: NgrokTunnel: "https://f635-130-211-249-200.ngrok-free.app" -> "http://localhost:8050"

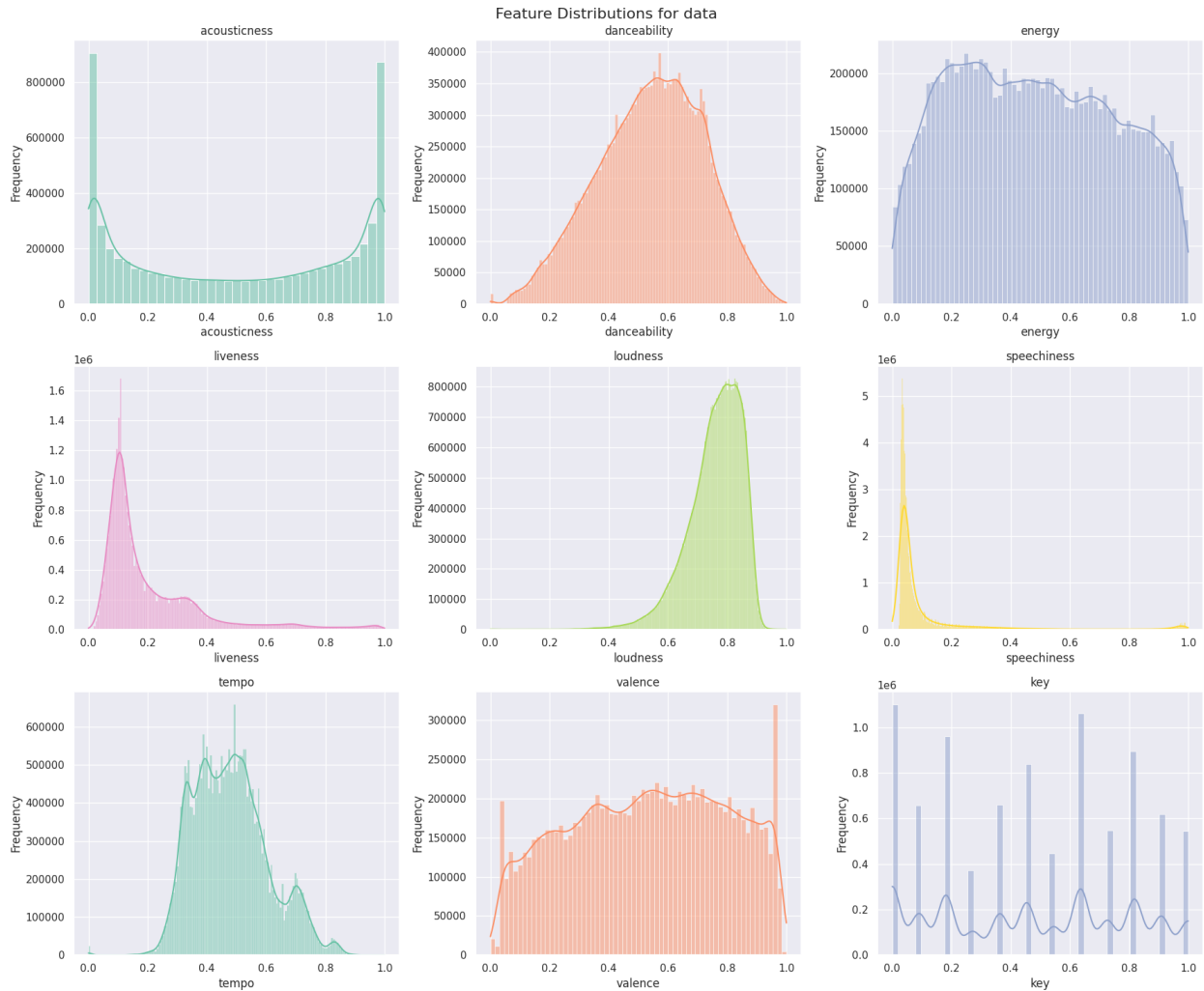
<IPython.core.display.Javascript object>
```

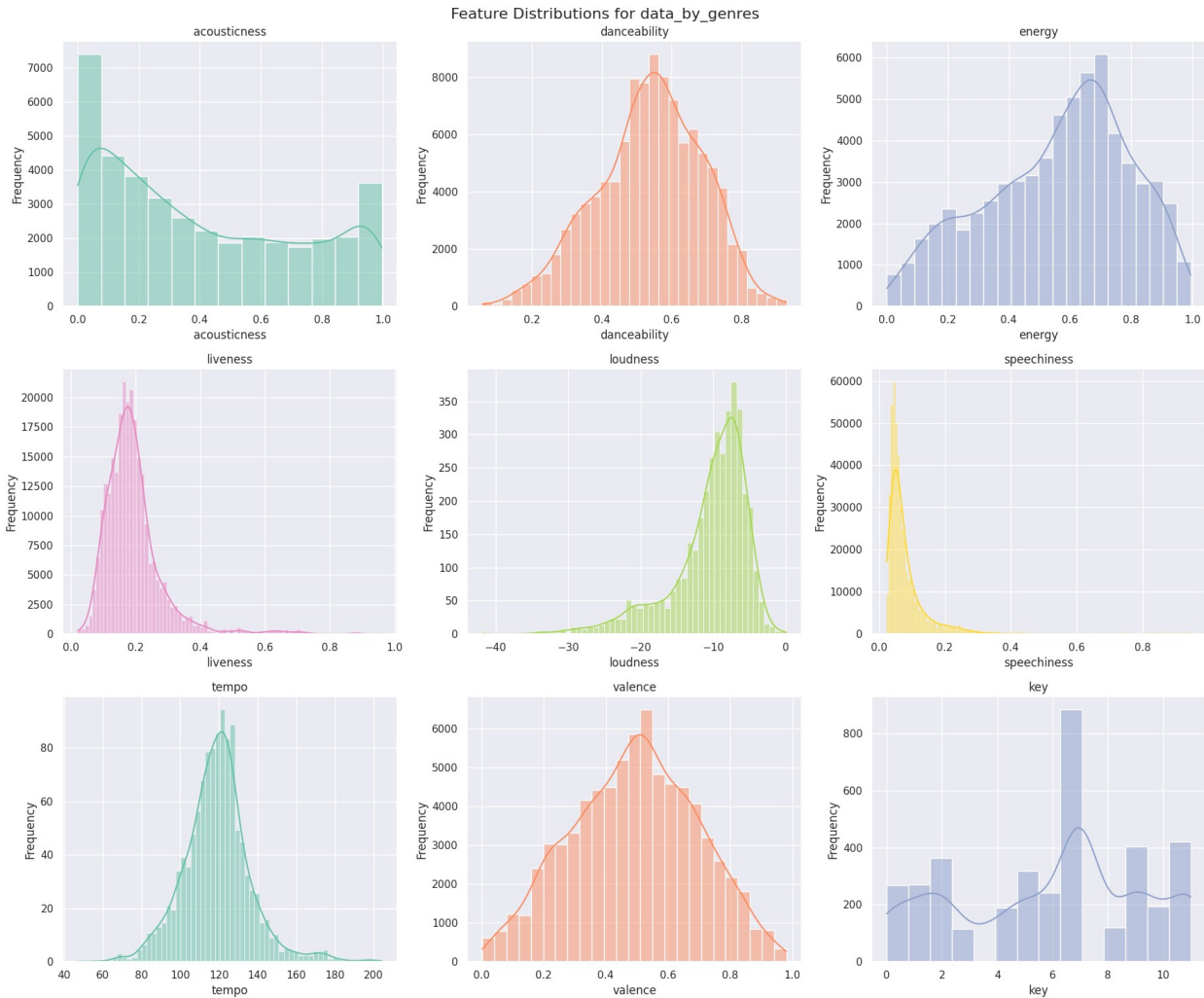
### Spotify Music Features Histogram

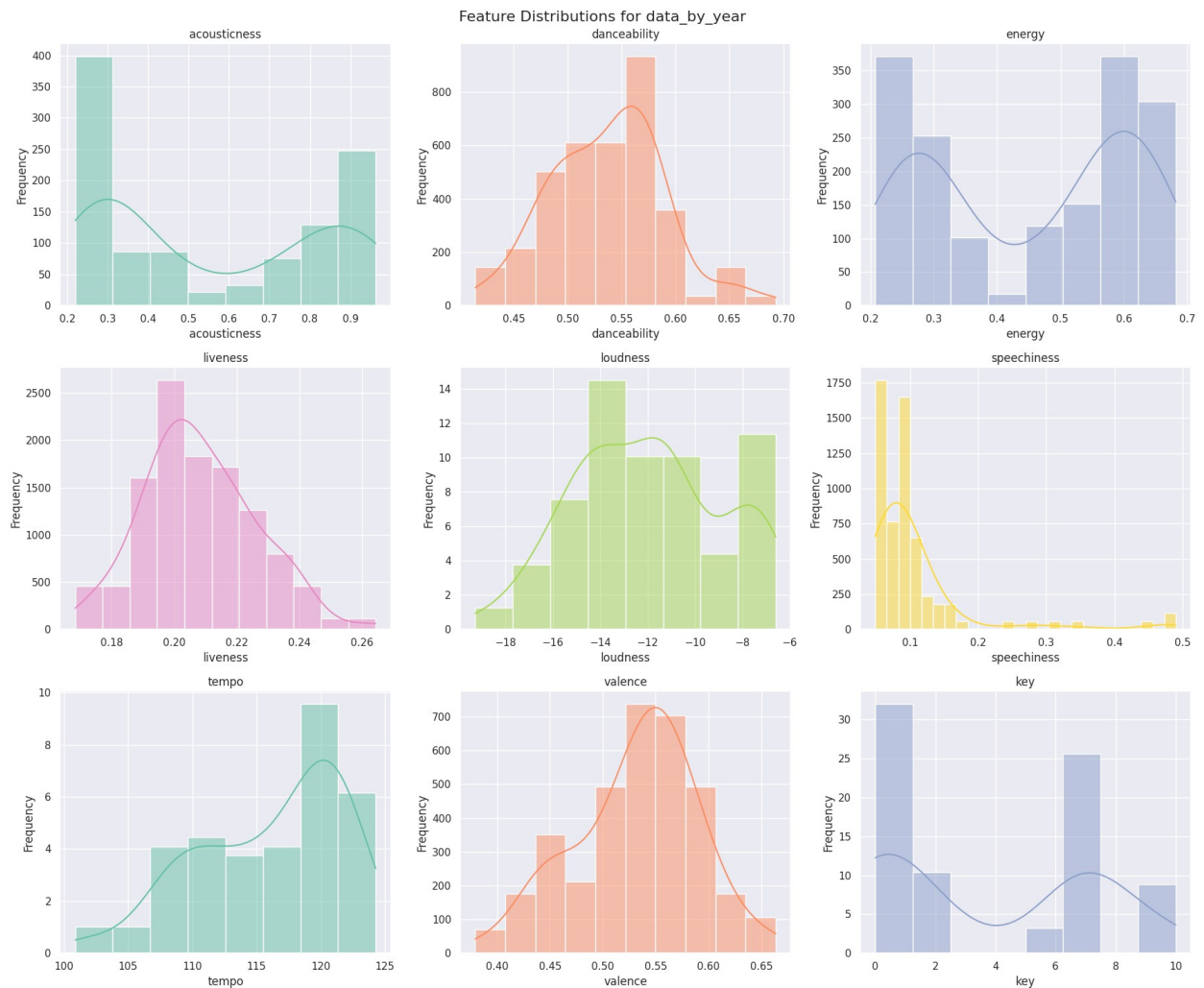
```
feature_names = ['acousticness', 'danceability', 'energy', 'liveness',
                 'loudness', 'speechiness', 'tempo', 'valence', 'key']

def plot_feature_distributions(df, name):
    plt.figure(figsize=(18, 15))
    plt.suptitle(f'Feature Distributions for {name}', fontsize=16)
    for i, feature in enumerate(feature_names):
        if feature in df.columns:
            plt.subplot(3, 3, i + 1)
            sns.histplot(df[feature], kde=True, stat='frequency',
                        color=sns.color_palette("Set2")[i % 6])
            plt.title(feature)
    plt.tight_layout()
    plt.show()

plot_feature_distributions(df, "data")
plot_feature_distributions(df_genres, "data_by_genres")
plot_feature_distributions(df_year, "data_by_year")
```

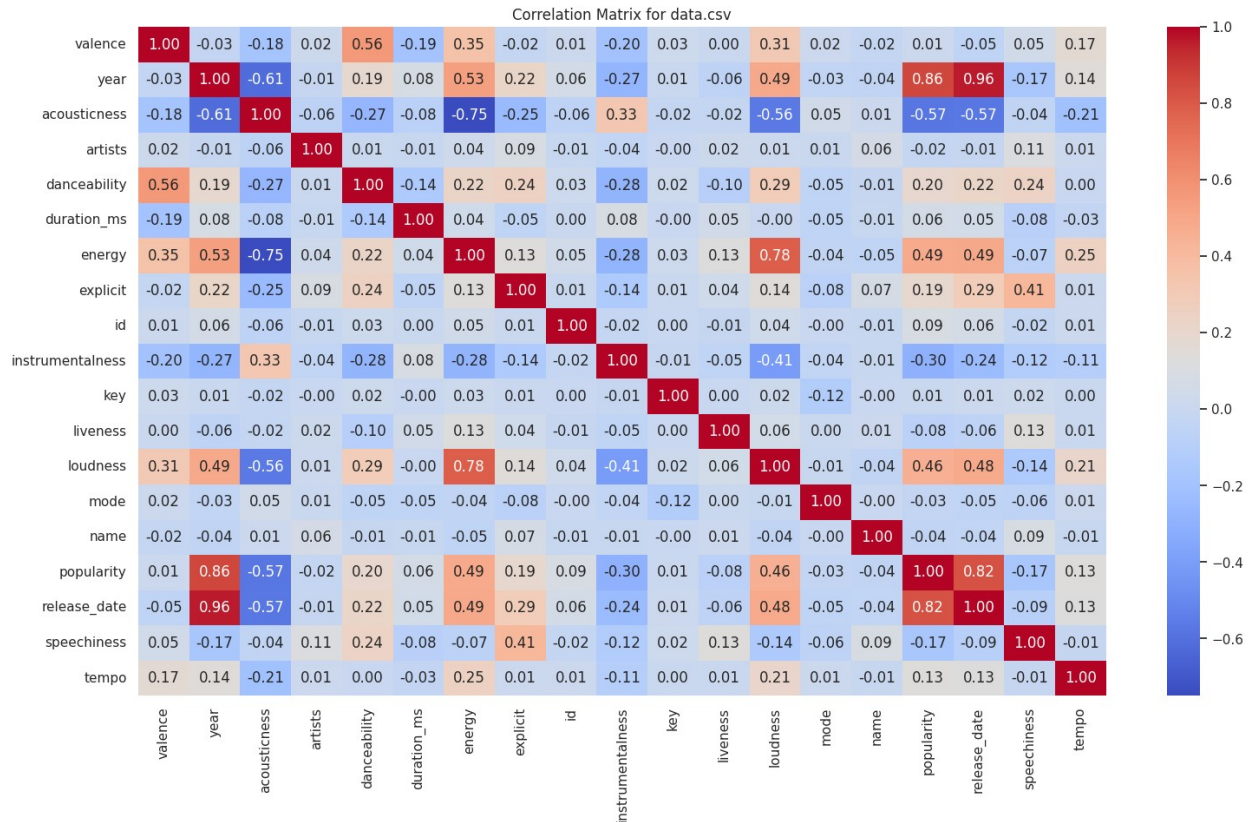






## Correlations Among Features Using Heatmaps

```
plt.figure(figsize=(18, 10))
numeric_df = df.select_dtypes(include=np.number)
corr_matrix = numeric_df.corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title(f'Correlation Matrix for data.csv')
plt.show()
```



## Feature Engineering

```
features_for_clustering = ['acousticness', 'danceability', 'energy',
                           'instrumentalness', 'liveness', 'valence', 'tempo']
```

```
X = df[features_for_clustering]
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

```
wcss = []
```

```
for i in range(1, 11):
```

```
    kmeans = KMeans(n_clusters=i, random_state=42)
```

```
    kmeans.fit(X_scaled)
```

```
    wcss.append(kmeans.inertia_)
```

```
plt.plot(range(1, 11), wcss)
```

```
plt.title('Elbow Method')
```

```
plt.xlabel('Number of clusters')
```

```
plt.ylabel('Within-Cluster Sum of Squares')
```

```
plt.show()
```

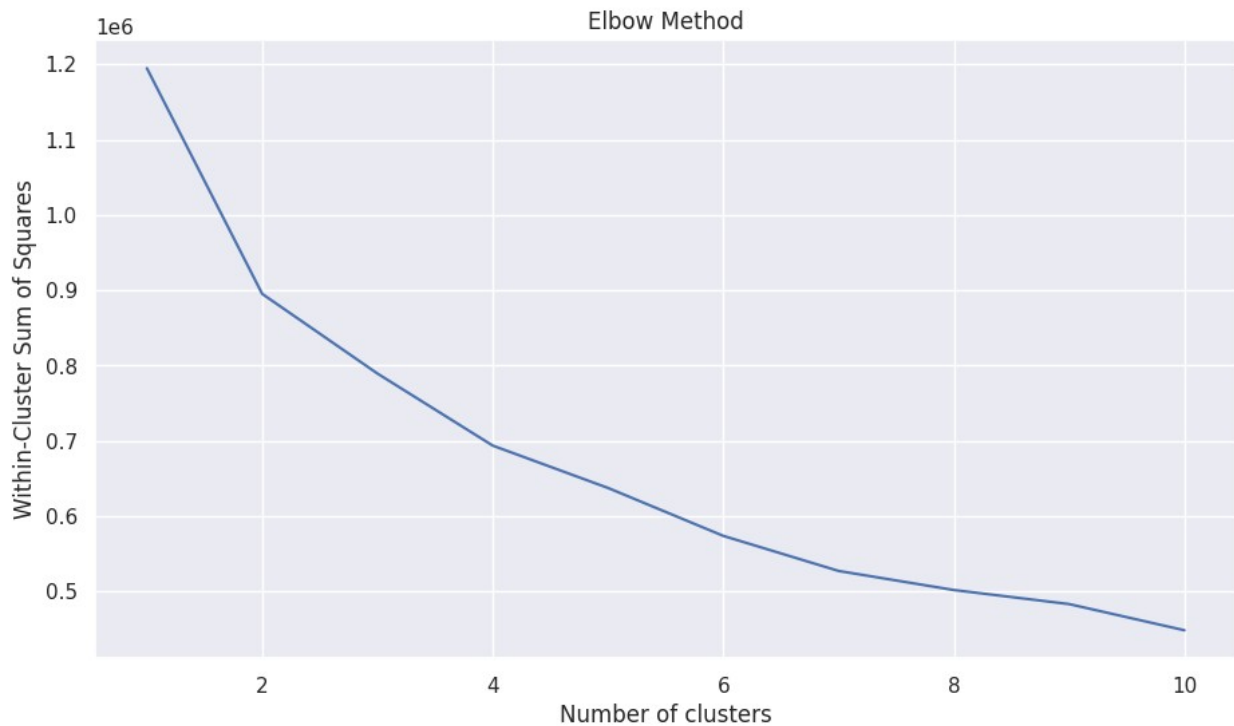
```
optimal_k = 2
```

```

kmeans = KMeans(n_clusters=optimal_k, random_state=42)
df['cluster'] = kmeans.fit_predict(X_scaled)

numerical_cols = df.select_dtypes(include=np.number).columns
print(df.groupby('cluster')[numerical_cols].mean())

```



|               | valence  | year     | acousticness | artists | danceability |
|---------------|----------|----------|--------------|---------|--------------|
| duration_ms \ |          |          |              |         |              |
| cluster       |          |          |              |         |              |
| 0             | 0.390545 | 0.411233 | 0.848392     | 0.48871 | 0.456016     |
| 0.041922      |          |          |              |         |              |
| 1             | 0.626967 | 0.672030 | 0.258784     | 0.51821 | 0.606572     |
| 0.041773      |          |          |              |         |              |

|            | energy   | explicit | id       | instrumentalness | key      |
|------------|----------|----------|----------|------------------|----------|
| liveness \ |          |          |          |                  |          |
| cluster    |          |          |          |                  |          |
| 0          | 0.245703 | 0.017603 | 0.483499 | 0.314335         | 0.463927 |
| 0.196555   |          |          |          |                  |          |
| 1          | 0.651070 | 0.132305 | 0.511760 | 0.062014         | 0.478975 |
| 0.212455   |          |          |          |                  |          |

|               | loudness | mode | name | popularity | release_date |
|---------------|----------|------|------|------------|--------------|
| speechiness \ |          |      |      |            |              |
| cluster       |          |      |      |            |              |

|          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|
| 0        | 0.697781 | 0.721526 | 0.505158 | 0.192963 | 0.192393 |
| 0.090721 |          |          |          |          |          |
| 1        | 0.804402 | 0.696480 | 0.493632 | 0.400805 | 0.463127 |
| 0.109073 |          |          |          |          |          |

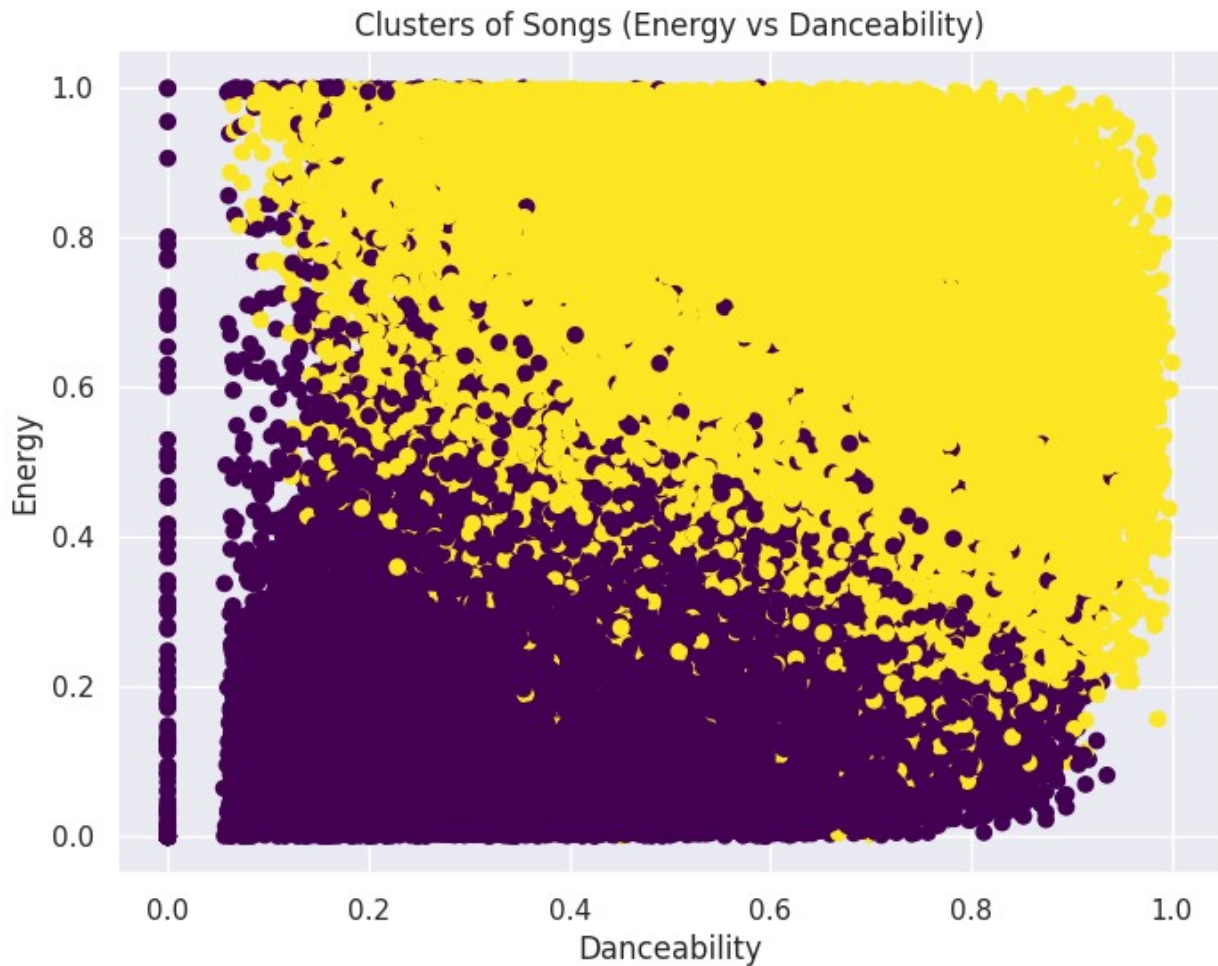
  

|         | tempo    | cluster |
|---------|----------|---------|
| cluster |          |         |
| 0       | 0.438856 | 0.0     |
| 1       | 0.509169 | 1.0     |

### Clustering Songs

```
plt.figure(figsize=(8, 6))
plt.scatter(df['danceability'], df['energy'], c=df['cluster'],
            cmap='viridis')
plt.xlabel('Danceability')
plt.ylabel('Energy')
plt.title('Clusters of Songs (Energy vs Danceability)')
plt.show()
```





### PCA Projection of Song Features

I can identify clusters of similar songs and how well the features differentiate, reveals overlaps or separations in song clusters.

```
X = df[features_for_clustering]

pca_pipeline = Pipeline([('scaler', StandardScaler()), ('PCA',
PCA(n_components=2))])
song_embedding = pca_pipeline.fit_transform(X)

projection = pd.DataFrame(columns=['PC1', 'PC2'], data=song_embedding)
projection['title'] = df['name']
projection['cluster'] = df['cluster']

fig = px.scatter(
    projection, x='PC1', y='PC2', color='cluster', hover_data=['PC1',
'PC2', 'title'],
    title='PCA Projection of Song Features' )
fig.show()
```

Visualize clusters

# Recommendation Model

## Collaborative Filtering

```
from sklearn.decomposition import TruncatedSVD
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split

df_collab = df[['id', 'name', 'popularity']].copy()
df_collab['user_id'] = df_collab.index % 50
df_collab = df_collab.rename(columns={'popularity': 'rating'})

interaction_matrix = df_collab.pivot(index='user_id', columns='id',
values='rating').fillna(0)

train_data, test_data = train_test_split(interaction_matrix,
test_size=0.2, random_state=42)

train_array = train_data.values
test_array = test_data.values

svd = TruncatedSVD(n_components=20, random_state=42)
svd.fit(train_array)

# Get predictions for the test data
predicted_matrix_test = np.dot(svd.transform(test_array),
svd.components_)

# Use the test data mask for RMSE calculation
test_mask = test_array > 0
rmse = np.sqrt(mean_squared_error(test_array[test_mask],
predicted_matrix_test[test_mask]))
print(f"RMSE: {rmse:.2f}")

# Get predictions for the entire interaction matrix (including train data)
predicted_matrix = np.dot(svd.transform(interaction_matrix.values),
svd.components_)

# Recommend songs for a user
def recommend_songs_for_user(user_id, top_n=5):
    if user_id in train_data.index:
        user_idx = train_data.index.get_loc(user_id)
        user_predictions = predicted_matrix[user_idx]
        song_ids = interaction_matrix.columns[np.argsort(-
user_predictions)[:top_n]]
        return df[df['id'].isin(song_ids)][['name', 'artists',
```

```

'popularity']]
    else:
        print(f"User ID {user_id} not found in training data.")
        return None

recommend_songs_for_user(user_id=10, top_n=5)

RMSE: 41.83

{"summary": "{\n  \"name\": \"recommend_songs_for_user(user_id=10,\n  top_n=5)\",\n  \"rows\": 5,\n  \"fields\": [\n    {\n      \"column\":\n      \"name\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 5,\n        \"samples\": [\n          \"Will\n          Anything Happen\",\n          \"Give Ireland Back To The Irish\",\n          \"Amor Del Bueno\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\":\n      \"artists\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 5,\n        \"samples\": [\n          \"['Blondie']\",\n          \"['Wings']\",\n          \"['Tito\n          Rojas']\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\":\n      \"popularity\",\n      \"properties\": {\n        \"dtype\":\n        \"number\",\n        \"std\": 11,\n        \"min\": 10,\n        \"max\": 42,\n        \"num_unique_values\": 5,\n        \"samples\":\n        [\n          30,\n          25,\n          34\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ],\n  \"type\": \"dataframe\"}"}

```

## Hybrid Model

```

def hybrid_recommendation(song_name, user_id, top_n=5):
    # Content-based filtering recommendations
    content_recs = recommend_songs(song_name, top_n=top_n)
    content_ids = content_recs['name'].values

    # Collaborative filtering recommendations
    collab_recs = recommend_songs_for_user(user_id, top_n=top_n)
    collab_ids = collab_recs['name'].values

    # Combine recommendations
    combined_recs = pd.concat([content_recs,
    collab_recs]).drop_duplicates(subset='name').head(top_n)
    return combined_recs

hybrid_recommendation(song_name='Shape of You', user_id=10, top_n=5)

{"summary": "{\n  \"name\": \"hybrid_recommendation(song_name='Shape of\n  You', user_id=10, top_n=5)\",\n  \"rows\": 5,\n  \"fields\": [\n    {\n      \"column\": \"name\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 5,\n        \"samples\": [\n          \"Shape of You\",\n          \"Shape of You\",\n          \"Shape of You\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ],\n  \"type\": \"dataframe\"}"}

```

```

\"samples\": [\n          \"Let Me Luv Your Girl\", \n          \"Poor Boy\", \n          \"Brujeria\" ], \n          \"semantic_type\": \"\", \n          \"description\": \"\" \n          }, \n          { \n          \"column\": \"artists\", \n          \"properties\": { \n          \"dtype\": \"string\", \n          \"num_unique_values\": 5, \n          \"samples\": [\n          \"['Mr. Capone-E']\", \n          \"['Howlin' Wolf']\", \n          \"['El Gran Combo De Puerto Rico']\" \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\" \n          }, \n          { \n          \"column\": \"popularity\", \n          \"properties\": { \n          \"dtype\": \"number\", \n          \"std\": 22, \n          \"min\": 23, \n          \"max\": 85, \n          \"num_unique_values\": 5, \n          \"samples\": [\n          41, \n          23, \n          58 \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\" \n          } \n          } \n          ], \"type\": \"dataframe\"}

```

```
!kill ngrok
```

```

def content_based_recommendation(song_name, data, features, top_n=5):
    if song_name not in data['track_name'].values:
        return []

    song_features = data[data['track_name'] == song_name]
    [features].values

    # Compute cosine similarity between the input song and all other songs
    similarity = cosine_similarity(song_features,
    data[features].values)

    # Get indices of the most similar songs
    similar_indices = np.argsort(similarity[0])[-(top_n + 1):-1][::-1]

    # Fetch song names for the most similar songs
    similar_songs = data.iloc[similar_indices]['track_name'].values

    return similar_songs

app = JupyterDash(__name__)

# Dash layout
app.layout = html.Div([
    html.H1("Music Recommendation System", style={'textAlign': 'center'}),
    html.Div([
        dcc.Input(
            id='input-song',
            type='text',
            placeholder='Enter a song name',
            style={'width': '50%', 'padding': '10px', 'margin': '10px auto', 'display': 'block'}

```

```

    ),
    html.Button(
        'Get Recommendations',
        id='recommend-button',
        n_clicks=0,
        style={'display': 'block', 'margin': '10px auto',
'padding': '10px'}
    ),
    html.Div(id='output-recommendations', style={'textAlign':
'center', 'marginTop': '20px'})
    ])
])

# Callback for recommendations
@app.callback(
    Output('output-recommendations', 'children'),
    [Input('recommend-button', 'n_clicks')],
    [Input('input-song', 'value')]
)
def update_recommendations(n_clicks, song_name):
    if n_clicks > 0 and song_name:
        recommendations = content_based_recommendation(song_name, df,
numeric_features)
        if recommendations:
            return html.Ul([html.Li(song) for song in
recommendations])
        else:
            return "Song not found in the dataset."
    return "Enter a song name and click the button to get
recommendations."

ngrok_tunnel = ngrok.connect(8052, bind_tls=True)
public_url = ngrok_tunnel.public_url
print(f"App is running at {public_url}")

app.run_server(mode="external", debug=True)
ngrok.kill()

```

## Content-Based Recommendation System

```

!kill ngrok
ngrok.set_auth_token("2rM2jNChPC6upKIyu1R0hywZIQe_5yJe2EtfCXUemirxV9aME")

def load_datasets():
    file_path1 = '/content/drive/My Drive/MusicData/data.csv'
    datasets = {
        "Play What Song You Want": pd.read_csv(file_path1),
        # "Genres": pd.read_csv(file_path2),
    }

```

```

        # "Year": pd.read_csv(file_path3),,
    }
    return datasets

# Preprocess datasets
def preprocess_datasets(datasets):
    scaler = MinMaxScaler()
    for name, dataset in datasets.items():
        dataset.fillna(dataset.median(numeric_only=True),
inplace=True)
        dataset.fillna("Unknown", inplace=True)
        numeric_cols =
dataset.select_dtypes(include=np.number).columns
        dataset[numeric_cols] =
scaler.fit_transform(dataset[numeric_cols])
    return datasets

# Initialize datasets
datasets = preprocess_datasets(load_datasets())

# Cluster songs using K-Means
def cluster_songs(dataset, num_clusters=10):
    feature_cols = ['acousticness', 'danceability', 'energy',
'instrumentalness', 'liveness', 'speechiness', 'valence', 'tempo']
    if set(feature_cols).issubset(dataset.columns):
        kmeans = KMeans(n_clusters=num_clusters, random_state=42)
        dataset['cluster'] = kmeans.fit_predict(dataset[feature_cols])
    return dataset

datasets["Play What Song You Want"] = cluster_songs(datasets["Play
What Song You Want"])

# Recommendation function
def recommend_songs(input_value, dataset, dataset_name,
num_recommendations=5):
    try:
        if dataset_name == "Play What Song You Want":
            dataset['name_normalized'] =
dataset['name'].str.lower().str.strip()
            input_value = input_value.lower().strip()

            if input_value not in dataset['name_normalized'].values:
                return f"Song '{input_value}' not found."

            song_index = dataset[dataset['name_normalized'] ==
input_value].index[0]
            feature_cols = ['acousticness', 'danceability', 'energy',
'instrumentalness', 'liveness', 'speechiness', 'valence', 'tempo']
            feature_matrix = dataset[feature_cols].values
            similarity_scores =

```

```

cosine_similarity([feature_matrix[song_index]], feature_matrix)[0]
    similar_indices = np.argsort(similarity_scores)[::-1]
[1:num_recommendations + 1]
    return dataset.iloc[similar_indices][['name', 'artists']]

    elif dataset_name == "Artists":
        return
dataset[dataset['artists'].str.contains(input_value, case=False,
na=False)].sample(n=num_recommendations)[['name', 'artists']]

    return f"Dataset '{dataset_name}' is not supported for
recommendations."
except Exception as e:
    return f"Error: {str(e)}"

# Dash app
app = Dash(__name__, external_stylesheets=[dbc.themes.CYBORG])
data_options = [{'label': name, 'value': name} for name in
datasets.keys()]

app.layout = dbc.Container([
    dbc.Row([
        dbc.Col(html.H1("♪ Music Recommendation System",
className="text-center mb-4"), width=12)
    ]),
    dbc.Row([
        dbc.Col([
            html.Label("Select Dataset:"),
            dcc.Dropdown(id='dataset-dropdown', options=data_options,
value='Play What Song You Want', className="mb-4"),
            html.Label("Song Title:"),
            dcc.Input(id='song-input', type='text', placeholder='e.g.,
Shape of You', className="mb-4"),
            dbc.Button('Recommend', id='recommend-button',
color="primary", className="mb-4")
        ], width=5),
        dbc.Col([
            html.Div(id='recommendations-output', className="mb-4"),
            html.Div(id='heatmap-output')
        ], width=8)
    ])
], fluid=True)

# Callback function
@app.callback([
    Output('recommendations-output', 'children'),
    Output('heatmap-output', 'children')
],
[Input('recommend-button', 'n_clicks'),
Input('song-input', 'value'),

```

```

        Input('dataset-dropdown', 'value')]
    )

    def update_output(n_clicks, input_value, dataset_name):
        if n_clicks and input_value:
            dataset = datasets[dataset_name]
            recommendations = recommend_songs(input_value, dataset,
            dataset_name)

            if isinstance(recommendations, str):
                recommendation_text = dbc.Alert(recommendations,
color="danger")
            else:
                recommendation_list = [dbc.ListGroupItem(f"{row['name']}"
by {row['artists']}") for _, row in recommendations.iterrows()]
                recommendation_text = dbc.ListGroup(recommendation_list,
flush=True)

                numeric_dataset = dataset.select_dtypes(include=np.number)
                if not numeric_dataset.empty:
                    heatmap_fig = px.imshow(numeric_dataset.corr(),
title=f"Correlation Heatmap for {dataset_name}", text_auto=True)
                    return recommendation_text, dcc.Graph(figure=heatmap_fig)
                return "", ""

    if __name__ == '__main__':
        app.run_server(debug=True, port=8052)

# # Open a tunnel to port 8053
    ngrok_tunnel = ngrok.connect(8052, bind_tls=True)
    public_url = ngrok_tunnel.public_url
    print(f"App is running at {public_url}")

# app.run_server(mode="external", debug=True)
#ngrok.kill()

<IPython.core.display.Javascript object>

App is running at https://5ba8-130-211-249-200.ngrok-free.app

```

## Evaluation

```

from sklearn.model_selection import train_test_split

df_pca = pd.DataFrame(data=df_pca, columns=['PC1', 'PC2'])
df_pca['target'] = df['explicit']
y = df['explicit']

X = df_pca[['PC1', 'PC2']]
y = df_pca['target']

```



```

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

model = LogisticRegression(max_iter=400)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
y_pred_prob = model.predict_proba(X_test)[:, 1]

# Calculate evaluation metrics
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
map_score = average_precision_score(y_test, y_pred_prob)

# Print metrics
print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")
print(f"F1-score: {f1:.2f}")
print(f"Mean Average Precision (MAP): {map_score:.2f}")

feature_cols = ['acousticness', 'danceability', 'energy',
'instrumentalness', 'liveness', 'speechiness', 'valence', 'tempo']
feature_matrix = df[feature_cols].values

# Find songs similar to a given song (index 0)
song_index = 0
similarity_scores = cosine_similarity([feature_matrix[song_index]],
feature_matrix)[0]

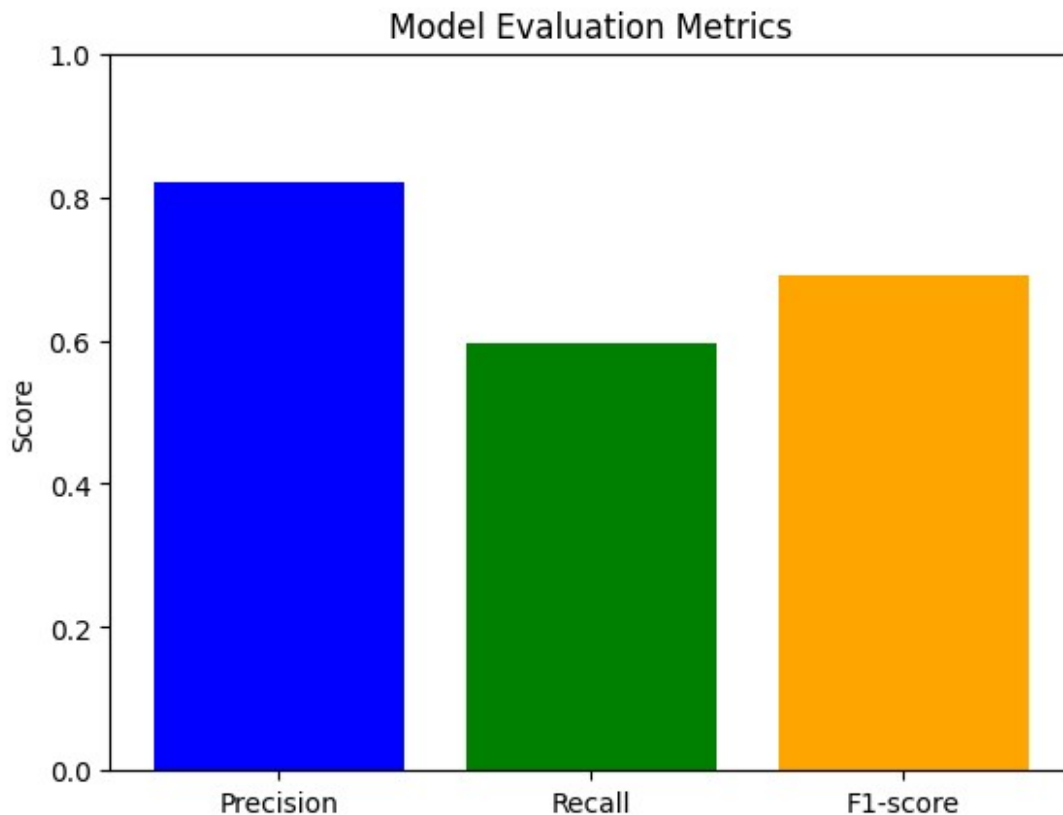
# Get top 5 recommendations
top_indices = np.argsort(similarity_scores)[::-1][1:6]
recommended_songs = df.iloc[top_indices][['name', 'artists']]
print("Recommended Songs:")
print(recommended_songs)

metrics = {'Precision': precision, 'Recall': recall, 'F1-score': f1}
plt.bar(metrics.keys(), metrics.values(), color=['blue', 'green',
'orange'])
plt.title('Model Evaluation Metrics')
plt.ylabel('Score')
plt.ylim(0, 1)
plt.show()

Precision: 0.82
Recall: 0.60
F1-score: 0.69
Mean Average Precision (MAP): 0.78
Recommended Songs:

```

|        | name     | artists  |
|--------|----------|----------|
| 39     | 0.625807 | 0.787368 |
| 1320   | 0.077059 | 0.309267 |
| 76360  | 0.161400 | 0.387068 |
| 156253 | 0.761241 | 0.394051 |
| 126084 | 0.451222 | 0.345704 |



## Visualization and Insights

```
def recommend_songs(df, song_name, top_n=5):
    try:
        song_name = song_name.lower()
        df['name_lower'] = df['name'].str.lower()

        song_idx = df[df['name_lower'] == song_name].index[0]
        audio_features = ['valence', 'acousticness', 'danceability',
                          'energy', 'instrumentalness', 'liveness', 'loudness', 'speechiness',
                          'tempo']
        song_vector =
df[audio_features].iloc[song_idx].values.reshape(1, -1)
        similarity = cosine_similarity(song_vector,
df[audio_features])
        similar_indices = similarity.argsort()[0][-top_n - 1:-1][::-1]
        recommendations = df.iloc[similar_indices][['name', 'artists',
```

```

'popularity']]
    return recommendations

    except IndexError:
        return f"Song '{song_name}' not found in the dataset."

def trending_alignment(df, song_name, top_n=5):
    try:
        song_name = song_name.lower()
        df['name_lower'] = df['name'].str.lower()

        song_popularity = df[df['name_lower'] == song_name]
        ['popularity'].iloc[0]
        trending_songs = df.nlargest(top_n, 'popularity')

        # Calculate the average popularity of the top trending songs
        avg_trending_popularity = trending_songs['popularity'].mean()

        if song_popularity > avg_trending_popularity:
            return f"Your preference for '{song_name}' aligns well
with trending songs. It's more popular than the average of the top
{top_n} trending songs."
        elif song_popularity == avg_trending_popularity:
            return f"Your preference for '{song_name}' is about average
compared to the top {top_n} trending songs."
        else:
            return f"Your preference for '{song_name}' is less aligned
with current trends. It's less popular than the average of the top
{top_n} trending songs."

    except IndexError:
        return f"Song '{song_name}' not found in the dataset."

recommendations = recommend_songs(df, "Shape of you")
print("Top 5 recommended songs for you:")
print(recommendations)

alignment_message = trending_alignment(df, "Shape of you")
alignment_message

```

Top 5 recommended songs for you:

|            | name                | artists                             |
|------------|---------------------|-------------------------------------|
| popularity |                     |                                     |
| 74616      | Shape of You        | ['Ed Sheeran']                      |
| 73         |                     |                                     |
| 108190     | Que Te Vaya Bien    | ['Grupo Jalado']                    |
| 65         |                     |                                     |
| 91998      | Lupe Campos         | ['El Fantasma', 'Los Dos Carnales'] |
| 68         |                     |                                     |
| 155281     | La Suerte del Señor | ['El Fantasma', 'Los Dos Carnales'] |

```

62
54849      Paz en Este Amor      ['Fidel Rueda']
61

{"type": "string"}

```

## Visualization

```

def plot_recommendations(recommendations, song_name):
    plt.figure(figsize=(10, 6))
    sns.barplot(x='popularity', y='name', data=recommendations,
palette='viridis', hue='name', dodge=False, legend=False)
    plt.title(f"Top Recommendations Similar to '{song_name}'",
fontsize=16)
    plt.xlabel("Popularity", fontsize=12)
    plt.ylabel("Songs", fontsize=12)
    plt.tight_layout()
    plt.show()

def plot_trending_alignment(df, song_name, top_n=5):
    trending_songs = df.nlargest(top_n, 'popularity')

    plt.figure(figsize=(10, 6))
    sns.barplot(x='popularity', y='name', data=trending_songs,
palette='coolwarm', hue='name', dodge=False, legend=False)
    plt.axvline(df[df['name'] == song_name]['popularity'].iloc[0],
color='green', linestyle='--', label=f"{song_name} Popularity")
    plt.title(f"Popularity of Top {top_n} Trending Songs vs
'{song_name}'", fontsize=16)
    plt.xlabel("Popularity", fontsize=12)
    plt.ylabel("Songs", fontsize=12)
    plt.legend()
    plt.tight_layout()
    plt.show()

recommendations = recommend_songs(df, "Shape of You")
if isinstance(recommendations, pd.DataFrame):
    plot_recommendations(recommendations, "Shape of You")

plot_trending_alignment(df, "Shape of You", top_n=5)

```

